

Sadece Bir Kez Bakarsın (You Only Look Once): Birleşik, Gerçek Zamanlı Nesne Tespiti

Joseph Redmon*, Santosh Divvala*†, Ross Girshick¶, Ali Farhadi*†
University of Washington*, Allen Institute for AI†, Facebook AI Research¶
<http://pjreddie.com/yolo/>

Özet

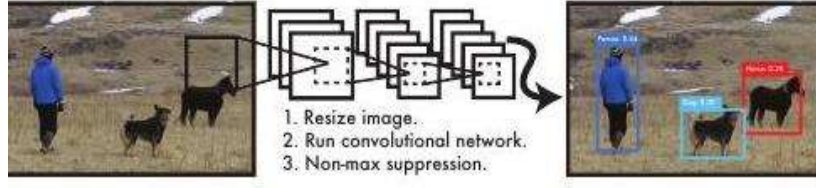
YOLO'yu, nesne tespiti (object detection) yönelik yeni bir yaklaşım olarak sunuyoruz. Nesne tespiti üzerine yapılan önceki çalışmalar, tespiti gerçekleştirmek için sınıflandırıcıları (classifiers) yeniden amaçlandırmaktadır (repurpose). Bunun yerine, biz nesne tespitini, uzamsal olarak ayrılmış bounding box'lara ve bunlarla ilişkili sınıf olasılıklarına (class probabilities) yönelik bir regresyon problemi olarak çerçeveleyiyoruz. Tek bir sinir ağı (neural network), bounding box'ları ve sınıf olasılıklarını, tam görüntülerden tek bir değerlendirme doğrudan tahmin eder. Tüm tespit hattı (pipeline) tek bir ağ olduğundan, doğrudan tespit performansı üzerinden uçtan uca (end-to-end) optimize edilebilir.

Birleşik mimarimiz son derece hızlıdır. Temel YOLO modelimiz, görüntüleri saniyede 45 kare (frames per second) hızında gerçek zamanlı olarak işler. Ağın daha küçük bir versiyonu olan Fast YOLO, diğer gerçek zamanlı dedektörlerin iki katı mAP değerine ulaşırken saniyede şaşırtıcı bir şekilde 155 kare işler. En gelişmiş (state-of-the-art) tespit sistemleriyle karşılaştırıldığında, YOLO daha fazla lokalizasyon hatası yapar ancak arka planda (background) yanlış pozitif (false positive) tahmin etme olasılığı daha düşüktür. Son olarak, YOLO nesnelerin çok genel temsillerini (representations) öğrenir. Doğal görüntülerden sanat eserleri gibi diğer alanlara (domains) genelleme yaparken DPM ve R-CNN dahil diğer tespit yöntemlerinin performansını aşar.

1. Giriş

İnsanlar bir görüntüye şöyle bir göz attıklarında, görüntüde hangi nesnelerin bulunduğunu, nerede olduklarını ve birbirleriyle nasıl etkileşime girdiklerini anında bilirler. İnsan görsel sistemi hızlı ve doğrudur; bu da bize araba kullanmak gibi karmaşık görevleri çok az bilinçli çaba ile gerçekleştirme imkânı tanır. Nesne tespiti için hızlı ve doğru algoritmalar, bilgisayarların özel sensörler olmadan araba kullanmasına, yardımcı cihazların gerçek zamanlı sahne bilgisini insan kullanıcılara aktarmasına ve genel amaçlı, duyarlı robotik sistemlerin potansiyelinin ortaya çıkmasına olanak sağlayabilir.

Mevcut tespit sistemleri, tespiti gerçekleştirmek için sınıflandırıcıları yeniden amaçlandırır. Bir nesneyi tespit etmek için bu sistemler, o nesneye ait bir sınıflandırıcıyı alır ve bunu bir test görüntüsündeki çeşitli konumlarda ve ölçeklerde değerlendirir. Deformable parts models (DPM) gibi sistemler, sınıflandırıcının görüntünün tamamı üzerinde eşit aralıklı konumlarda çalıştırıldığı bir kayan pencere (sliding window) yaklaşımı kullanır [10].



Şekil 1: YOLO Tespit Sistemi. YOLO ile görüntü işleme basit ve anlaşılırdır. Sistemimiz (1) girdi görüntüsünü 448×448 boyutuna yeniden ölçeklendirir (resize), (2) görüntü üzerinde tek bir convolutional network çalıştırır ve (3) ortaya çıkan tespitleri modelin confidence değerine göre eşikler (threshold).

R-CNN gibi daha yeni yaklaşımlar, önce bir görüntüde olası bounding box'lar üretmek ve ardından bu önerilen box'lar üzerinde bir sınıflandırıcı çalıştırmak için bölge önerisi (region proposal) yöntemlerini kullanır. Sınıflandırmadan sonra, bounding box'ları iyileştirmek, yinelenen tespitleri elemek ve sahnedeki diğer nesnelere göre box'ları yeniden puanlamak için son işleme (post-processing) uygulanır [13]. Bu karmaşık pipeline'lar yavaşdır ve optimize edilmesi zordur, çünkü her bir bileşenin ayrı ayrı eğitilmesi gerekir.

Biz nesne tespitini, görüntü piksellerinden doğrudan bounding box koordinatlarına ve sınıf olasılıklarına giden tek bir regresyon problemi olarak yeniden çerçeveleyoruz. Sistemimizi kullanarak, bir görüntüye hangi nesnelerin mevcut olduğunu ve nerede bulunduklarını tahmin etmek için sadece bir kez bakarsınız (you only look once - YOLO).

YOLO ferahlatıcı derecede basittir: bkz. Şekil 1. Tek bir convolutional network, birden çok bounding box'ı ve bu box'lara ait sınıf olasılıklarını eş zamanlı olarak tahmin eder. YOLO, tam görüntüler üzerinde eğitilir ve tespit performansını doğrudan optimize eder. Bu birleşik model, geleneksel nesne tespiti yöntemlerine göre çeşitli avantajlar sunar.

Birincisi, YOLO son derece hızlıdır. Tespiti bir regresyon problemi olarak çerçevelediğimiz için karmaşık bir pipeline'a ihtiyacımız yoktur. Test zamanında tespitleri tahmin etmek için sinir ağıımızı yeni bir görüntü üzerinde çalıştırmamız yeterlidir. Temel ağıımız, bir Titan X GPU üzerinde batch işleme yapılmadan saniyede 45 kare hızında çalışır ve hızlı versiyon 150 fps'nin üzerinde çalışır. Bu, akan videoyu 25 milisaniyeden daha az bir gecikmeyle (latency) gerçek zamanlı olarak işleyebileceğimiz anlamına gelir. Ayrıca YOLO, diğer gerçek zamanlı sistemlerin ortalama hassasiyetinin (mean average precision) iki katından fazlasına ulaşır. Sistemimizin bir webcam üzerinde gerçek zamanlı çalışan bir demosu için lütfen proje web sayfamıza bakın: <http://pjreddie.com/yolo/>.

İkincisi, YOLO tahminler yaparken görüntü hakkında küresel (global) olarak akıl yürütür. Kayan pencere ve bölge önerisi tabanlı tekniklerin aksine, YOLO eğitim ve test zamanında görüntünün tamamını görür; bu sayede sınıflar hakkındaki bağlamsal bilgiyi (contextual information), görünümüleriyle birlikte örtük olarak kodlar. Üst düzey bir tespit yöntemi olan Fast R-CNN [14], daha geniş bağlamı göremediği için görüntüdeki arka plan (background) yamalarını nesne sanır. YOLO, Fast R-CNN ile karşılaştırıldığında yarisından daha az sayıda arka plan hatası yapar.

Üçüncüsü, YOLO nesnelerin genellenebilir temsillerini öğrenir. Doğal görüntüler üzerinde eğitilip sanat eserleri üzerinde test edildiğinde, YOLO DPM ve R-CNN gibi üst düzey tespit yöntemlerinin performansını büyük bir farkla aşar. YOLO oldukça genellenebilir olduğundan, yeni alanlara (domains) veya beklenmedik girdilere uygulandığında bozulma olasılığı daha düşüktür.

YOLO, doğruluk (accuracy) açısından hâlâ en gelişmiş tespit sistemlerinin gerisindedir. Görüntülerdeki nesneleri hızlıca tanımlayabilse de, özellikle küçük nesneler olmak üzere bazı nesneleri hassas bir şekilde lokalize etmekte zorlanır. Bu ödünleşimleri (tradeoffs) deneylerimizde daha ayrıntılı olarak inceliyoruz.

Tüm eğitim ve test kodumuz açık kaynaklıdır (open source). Çeşitli önceden eğitilmiş (pretrained) modeller de indirilmeye hazır durumdadır.

2. Birleşik Tespit

Nesne tespitinin ayrı bileşenlerini tek bir sinir ağında birleştiriyoruz. Ağımız, her bir bounding box'ı tahmin etmek için görüntünün tamamından elde edilen özellikleri (features) kullanır. Ayrıca bir görüntü için tüm sınıflardaki tüm bounding box'ları eş zamanlı olarak tahmin eder. Bu, ağımızın tüm görüntü ve görüntüdeki tüm nesneler hakkında küresel olarak akıl yürüttüğü anlamına gelir. YOLO tasarımı, yüksek ortalama hassasiyeti korurken uçtan uca eğitim ve gerçek zamanlı hızları mümkün kılar.

Sistemimiz, girdi görüntüsünü bir $S \times S$ grid'e böler. Bir nesnenin merkezi bir grid cell'in içine düşerse, o grid cell o nesneyi tespit etmekten sorumludur.

Her bir grid cell, B adet bounding box ve bu box'lar için confidence score'lar tahmin eder. Bu confidence score'lar, modelin box'ın bir nesne içerdiğine dair ne kadar emin olduğunu ve tahmin ettiği box'ın ne kadar doğru olduğunu düşündüğünü yansıtır. Biçimsel olarak confidence'ı aşağıdaki gibi tanımlıyoruz:

$$\Pr(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}}$$

Eğer o cell'de bir nesne yoksa, confidence score'lar sıfır olmalıdır. Aksi takdirde, confidence score'un tahmin edilen box ile ground truth arasındaki intersection over union (IOU) değerine eşit olmasını isteriz.

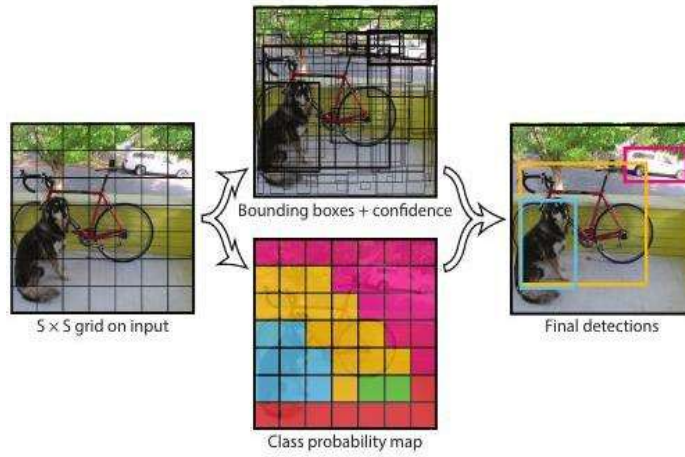
Her bounding box, 5 tahminden oluşur: x, y, w, h ve confidence. (x, y) koordinatları, box'ın merkezinin grid cell'in sınırlarına göre konumunu temsil eder. Genişlik (width) ve yükseklik (height), görüntünün tamamına göre tahmin edilir. Son olarak confidence tahmini, tahmin edilen box ile herhangi bir ground truth box arasındaki IOU'yu temsil eder.

Her grid cell ayrıca C adet koşullu (conditional) sınıf olasılığı, $\Pr(\text{Class}_i | \text{Object})$, tahmin eder. Bu olasılıklar, grid cell'in bir nesne içermesi koşuluna bağlıdır. B box sayısından bağımsız olarak, grid cell başına yalnızca bir set sınıf olasılığı tahmin ederiz.

Test zamanında, koşullu sınıf olasılıklarını ve bireysel box confidence tahminlerini çarpabiliriz:

$$\Pr(\text{Class}_i | \text{Object}) * \Pr(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}} = \Pr(\text{Class}_i) * \text{IOU}_{\text{pred}}^{\text{truth}} \quad (1)$$

bu da bize her box için sınıfa özgü (class-specific) confidence score'lar verir. Bu score'lar hem o sınıfın box içinde görünme olasılığını hem de tahmin edilen box'ın nesneye ne kadar iyi uyduğunu kodlar.



Şekil 2: Model. Sistemimiz tespiti bir regresyon problemi olarak modeller. Görüntüyü bir $S \times S$ grid'e böler ve her grid cell için B bounding box, bu box'lar için confidence ve C sınıf olasılığı tahmin eder. Bu tahminler, $S \times S \times (B * 5 + C)$ boyutunda bir tensor olarak kodlanır.

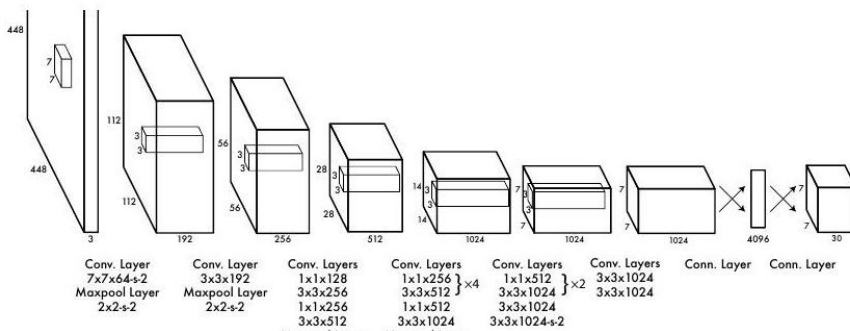
YOLO'yu PASCAL VOC üzerinde değerlendirmek için $S = 7$, $B = 2$ kullanıyoruz. PASCAL VOC'un 20 etiketli sınıfı vardır, dolayısıyla $C = 20$ 'dir. Nihai tahminimiz $7 \times 7 \times 30$ boyutunda bir tensor'dür.

2.1. Ağ Tasarımı

Bu modeli bir convolutional neural network olarak uyguluyor ve PASCAL VOC tespit veri kümesi (dataset) [9] üzerinde değerlendiriyoruz. Ağın ilk convolutional layer'ları görüntüden özellik (feature) çıkarırken, fully connected layer'lar çıktı olasılıklarını ve koordinatlarını tahmin eder.

Ağ mimarimiz, görüntü sınıflandırma (image classification) için kullanılan GoogLeNet modelinden [34] esinlenmiştir. Ağımız, ardından 2 fully connected layer gelen 24 convolutional layer'a sahiptir. GoogLeNet'in kullandığı inception module'leri yerine, Lin ve ark. [22] çalışmasına benzer şekilde, ardından 3×3 convolutional layer gelen basit 1×1 reduction layer'lar kullanıyoruz. Ağın tamamı Şekil 3'te gösterilmiştir.

Ayrıca hızlı nesne tespitinin sınırlarını zorlamak üzere tasarlanmış, YOLO'nun hızlı bir versiyonunu da eğitiyoruz. Fast YOLO, daha az sayıda convolutional layer (24 yerine 9) ve bu layer'larda daha az filtre içeren bir sinir ağı kullanır. Ağın boyutu dışında, YOLO ve Fast YOLO arasındaki tüm eğitim ve test parametreleri aynıdır.



Şekil 3: Mimari. Tespit ağımız, ardından 2 fully connected layer gelen 24 convolutional layer'a sahiptir. Değişimli (alternating) 1×1 convolutional layer'lar, önceki layer'lardan gelen özellik uzayını (feature space) azaltır. Convolutional layer'ları, ImageNet

sınıflandırma görevinde yarı çözünürlükte (224×224 girdi görüntüsü) önceden eğitiliyor (pretrain) ve ardından tespit için çözünürlüğü iki katına çıkarıyoruz.

Ağımızın nihai çıktısı, tahminlerden oluşan $7 \times 7 \times 30$ boyutunda bir tensor'dür.

2.2. Eğitim

Convolutional layer'larımızı, ImageNet 1000 sınıflı yarışma veri kümesi [30] üzerinde önceden eğitiyoruz. Ön eğitim (pretraining) için, Şekil 3'teki ilk 20 convolutional layer'ı, ardından bir average-pooling layer ve bir fully connected layer kullanıyoruz. Bu ağı yaklaşık bir hafta boyunca eğitiyor ve ImageNet 2012 validation kümesinde, Caffe'nin Model Zoo'sundaki [24] GoogLeNet modelleriyle kıyaslanabilir düzeyde, tek crop için %88 top-5 accuracy elde ediyoruz. Tüm eğitim ve çıkarım (inference) için Darknet framework'ünü [26] kullanıyoruz.

Ardından modeli tespit gerçekleştirecek şekilde dönüştürüyoruz. Ren ve ark., pretrained ağlara hem convolutional hem de connected layer eklemenin performansı artırabileceğini gösterir [29]. Onların örneğini takip ederek, rastgele başlatılmış (randomly initialized) ağırlıklara sahip dört convolutional layer ve iki fully connected layer ekliyoruz. Tespit genellikle ince taneli (fine-grained) görsel bilgi gerektirdiğinden, ağı girdi çözünürlüğünü 224×224 'ten 448×448 'e çıkarıyoruz.

Son layer'ımız hem sınıf olasılıklarını hem de bounding box koordinatlarını tahmin eder. Bounding box genişliğini ve yüksekliğini, 0 ile 1 arasında olacak şekilde görüntü genişliğine ve yüksekliğine göre normalize ediyoruz. Bounding box'ın x ve y koordinatlarını, belirli bir grid cell konumunun offset'leri olacak şekilde parametrize ediyoruz; böylece bunlar da 0 ile 1 arasında sınırlı olur.

Son layer için doğrusal (linear) bir aktivasyon fonksiyonu kullanıyoruz; diğer tüm layer'lar ise aşağıdaki leaky rectified linear aktivasyonu kullanır:

$$\begin{aligned} \phi(x) &= x, & \text{eğer } x > 0 \\ \phi(x) &= 0.1x, & \text{diğer durumlarda} \end{aligned} \quad (2)$$

Modelimizin çıktısındaki sum-squared error'u optimize ediyoruz. Sum-squared error kullanıyoruz çünkü optimize edilmesi kolaydır; ancak bu, average precision'ı maksimize etme hedefimizle tam olarak örtüşmez. Lokalizasyon hatasını, ideal olmayabilecek şekilde sınıflandırma hatasıyla eşit ağırlıklandırır. Ayrıca her görüntüde birçok grid cell herhangi bir nesne içermez. Bu durum, o cell'lerin "confidence" score'larını sıfıra doğru iter ve genellikle nesne içeren cell'lerden gelen gradyanı bastırır. Bu, model kararsızlığına (instability) yol açarak eğitimin erken aşamada ıraksamasına (diverge) neden olabilir.

Bunu düzeltmek için, bounding box koordinat tahminlerinden gelen loss'u artırıyor ve nesne içermeyen box'lar için confidence tahminlerinden gelen loss'u azaltıyoruz. Bunu gerçekleştirmek için λ_{coord} ve λ_{noobj} adında iki parametre kullanıyoruz. $\lambda_{\text{coord}} = 5$ ve $\lambda_{\text{noobj}} = 0,5$ olarak belirliyoruz.

Sum-squared error, büyük box'lardaki ve küçük box'lardaki hataları da eşit şekilde ağırlıklandırır. Hata metriğimizin, büyük box'lardaki küçük sapmaların küçük box'lardakinden daha az önemli olduğunu yansıtması gerekir. Bunu kısmen ele almak için, bounding box genişliğini ve yüksekliğini doğrudan değil, karekökünü (square root) tahmin ediyoruz.

YOLO, her grid cell için birden çok bounding box tahmin eder. Eğitim zamanında, her nesne için yalnızca bir bounding box predictor'ın sorumlu olmasını isteriz. Ground truth ile en yüksek güncel IOU'ya sahip tahmine göre bir predictor'ı o nesneyi tahmin etmekten "sorumlu" olarak atarız. Bu, bounding box predictor'lar arasında

uzmanlaşmaya (specialization) yol açar. Her predictor, belirli boyutları, en-boy oranlarını (aspect ratios) veya nesne sınıflarını tahmin etmede daha iyi hale gelir ve bu da genel recall'ı artırır.

Eğitim sırasında, aşağıdaki çok parçalı (multi-part) loss fonksiyonunu optimize ediyoruz:

$$\begin{aligned} & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{obj}} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2] \\ & + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{obj}} [(vw_i - \hat{vw}_i)^2 + (vh_i - \hat{vh}_i)^2] \\ & + \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\ & + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\ & + \sum_{i=0}^{S^2} 1_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \quad (3) \end{aligned}$$

burada 1_i^{obj} , i hücresinde bir nesnenin bulunup bulunmadığını; 1_{ij}^{obj} ise i hücresindeki j. bounding box predictor'ının bu tahminden “sorumlu” olduğunu belirtir.

Loss fonksiyonunun, yalnızca o grid cell'de bir nesne mevcutsa sınıflandırma hatasını cezalandırdığına dikkat edin (bu nedenle daha önce bahsedilen koşullu sınıf olasılığı söz konusudur). Ayrıca yalnızca ilgili predictor ground truth box'tan “sorumlu” ise (yani o grid cell'deki tüm predictor'lar arasında en yüksek IOU'ya sahipse) bounding box koordinat hatasını cezalandırır.

Ağı, PASCAL VOC 2007 ve 2012'nin training ve validation veri kümeleri üzerinde yaklaşık 135 epoch boyunca eğitiyoruz. 2012 üzerinde test ederken, eğitime VOC 2007 test verisini de dahil ediyoruz. Eğitim boyunca 64'lük bir batch size, 0,9'luk bir momentum ve 0,0005'lik bir decay kullanıyoruz.

Learning rate programımız şu şekildedir: İlk epoch'larda learning rate'i 10^{-3} 'ten 10^{-2} 'ye yavaşça yükseltiyoruz. Yüksek bir learning rate ile başlarsak, modelimiz kararsız gradyanlar (unstable gradients) nedeniyle sıklıkla ıraksar. Eğitime 75 epoch boyunca 10^{-2} ile, ardından 30 epoch boyunca 10^{-3} ile ve son olarak 30 epoch boyunca 10^{-4} ile devam ediyoruz.

Overfitting'i önlemek için dropout ve yoğun data augmentation kullanıyoruz. İlk connected layer'dan sonra rate = 0,5 olan bir dropout layer, layer'lar arasındaki co-adaptation'ı önler [18]. Data augmentation için, orijinal görüntü boyutunun %20'sine kadar rastgele ölçekleme (scaling) ve öteleme (translation) uyguluyoruz. Ayrıca HSV renk uzayında görüntünün exposure ve saturation değerlerini 1,5 katına kadar rastgele ayarlıyoruz.

2.3. Çıkarım (Inference)

Eğitimde olduğu gibi, bir test görüntüsü için tespitleri tahmin etmek de yalnızca tek bir ağı değerlendirmesi gerektirir. PASCAL VOC üzerinde ağı, görüntü başına 98 bounding box ve her box için sınıf olasılıkları tahmin eder. YOLO, sınıflandırıcı tabanlı yöntemlerin aksine yalnızca tek bir ağı değerlendirmesi gerektirdiğinden, test zamanında son derece hızlıdır.

Grid tasarımı, bounding box tahminlerinde uzamsal çeşitliliği (spatial diversity) zorunlu kılar. Çoğu zaman bir nesnenin hangi grid cell'e düştüğü açıktır ve ağı her nesne için yalnızca bir box tahmin eder. Ancak bazı büyük nesneler veya birden fazla cell'in sınırına yakın nesneler, birden çok cell tarafından iyi bir şekilde lokalize edilebilir. Bu çoklu tespitleri düzeltmek için non-maximal suppression kullanılabilir. R-CNN veya DPM'de olduğu kadar performans için kritik olmasa da, non-maximal suppression mAP'a %2-3 katkı sağlar.

2.4. YOLO'nun Sınırlamaları

YOLO, her grid cell yalnızca iki box tahmin edebildiğinden ve yalnızca bir sınıfa sahip olabildiğinden, bounding box tahminlerine güçlü uzamsal kısıtlamalar (spatial constraints) getirir. Bu uzamsal kısıtlama, modelimizin tahmin edebileceği yakın nesnelerin sayısını sınırlar. Modelimiz, kuş sürüleri gibi grup hâlinde bulunan küçük nesnelerle zorlanır.

Modelimiz bounding box'ları veriden tahmin etmeyi öğrendiğinden, yeni veya alışılmadık en-boy oranlarına (aspect ratios) ya da konfigürasyonlara sahip nesnelere genelleme yapmakta zorlanır. Mimarimiz, girdi görüntüsünden itibaren birden çok downsampling layer içerdiğinden, modelimiz bounding box'ları tahmin etmek için nispeten kaba (coarse) özellikler de kullanır.

Son olarak, tespit performansına yaklaşan bir loss fonksiyonu üzerinde eğitim yapsak da, loss fonksiyonumuz küçük bounding box'lardaki ve büyük bounding box'lardaki hataları aynı şekilde ele alır. Büyük bir box'taki küçük bir hata genellikle zararsızdır, ancak küçük bir box'taki küçük bir hata IOU üzerinde çok daha büyük bir etkiye sahiptir. Ana hata kaynağımız yanlış lokalizasyonlardır.

3. Diğer Tespit Sistemleriyle Karşılaştırma

Nesne tespiti, computer vision alanında temel bir problemdir. Tespit pipeline'ları genellikle girdi görüntülerinden sağlam (robust) bir dizi özellik çıkararak başlar (Haar [25], SIFT [23], HOG [4], convolutional özellikler [6]). Ardından, özellik uzayında (feature space) nesneleri tanımlamak için sınıflandırıcılar [36, 21, 13, 10] veya lokalizatörler [1, 32] kullanılır. Bu sınıflandırıcılar veya lokalizatörler, ya görüntünün tamamı üzerinde sliding window şeklinde ya da görüntüdeki bazı bölge alt kümeleri üzerinde çalıştırılır [35, 15, 39]. YOLO tespit sistemini, temel benzerlikleri ve farkları vurgulayarak birkaç üst düzey tespit framework'üyle karşılaştırıyoruz.

Deformable parts models.

Deformable parts models (DPM), nesne tespitine sliding window yaklaşımı kullanır [10]. DPM; statik özellikler çıkarmak, bölgeleri sınıflandırmak, yüksek puanlı bölgeler için bounding box'lar tahmin etmek vb. için ayrık (disjoint) bir pipeline kullanır. Sistemimiz, bu birbirinden kopuk tüm parçaları tek bir convolutional neural network ile değiştirir. Ağ; özellik çıkarımını, bounding box tahminini, non-maximal suppression'ı ve bağlamsal akıl yürütmeyi (contextual reasoning) eş zamanlı olarak gerçekleştirir. Statik özellikler yerine ağ, özellikleri iç içe (in-line) eğitir ve bunları tespit görevi için optimize eder. Birleşik mimarimiz, DPM'den daha hızlı ve daha doğru bir model ortaya çıkarır.

R-CNN.

R-CNN ve varyantları, görüntülerdeki nesneleri bulmak için sliding window yerine bölge önerileri (region proposals) kullanır. Selective Search [35] olası bounding box'lar üretir, bir convolutional network özellik çıkarır, bir SVM box'ları puanlar, doğrusal (linear) bir model bounding box'ları ayarlar ve non-max suppression yinelenen tespitleri eler. Bu karmaşık pipeline'ın her aşamasının bağımsız olarak hassas biçimde ayarlanması gerekir ve ortaya çıkan sistem, test zamanında görüntü başına 40 saniyeden fazla sürerek çok yavaştır [14].

YOLO, R-CNN ile bazı benzerlikler taşır. Her grid cell, olası bounding box'lar önerir ve bu box'ları convolutional özellikler kullanarak puanlar. Ancak sistemimiz, grid cell önerilerine uzamsal kısıtlamalar getirir; bu da aynı nesnenin birden çok kez tespit edilmesini azaltmaya yardımcı olur. Sistemimiz ayrıca çok daha az sayıda bounding box önerir; Selective Search'ün önerdiği yaklaşık 2000 box'a karşılık görüntü başına yalnızca 98 box. Son olarak sistemimiz, bu ayrı bileşenleri tek, birlikte optimize edilmiş (jointly optimized) bir model hâlinde birleştirir.

Diğer Hızlı Dedektörler (Other Fast Detectors).

Fast ve Faster R-CNN, hesaplamayı paylaşarak ve bölge önermek için Selective Search yerine sinir ağıları kullanarak R-CNN framework'ünü hızlandırmaya odaklanır [14] [28]. R-CNN'e kıyasla hız ve doğruluk iyileştirmeleri sunsalar da, ikisi de hâlâ gerçek zamanlı performansın gerisinde kalır.

Birçok araştırma çalışması, DPM pipeline'ını hızlandırmaya odaklanır [31] [38] [5]. Bunlar HOG hesaplamasını hızlandırır, cascade'ler kullanır ve hesaplamayı GPU'lara taşır. Ancak yalnızca 30Hz DPM [31] gerçekten gerçek zamanlı çalışır.

Büyük bir tespit pipeline'ının tek tek bileşenlerini optimize etmeye çalışmak yerine, YOLO pipeline'ı tamamen ortadan kaldırır ve tasarım gereği hızlıdır.

Yüz veya insan gibi tek sınıflar için dedektörler, çok daha az varyasyonla uğraşmaları gerektiğinden yüksek düzeyde optimize edilebilir [37]. YOLO ise çeşitli nesneleri eş zamanlı olarak tespit etmeyi öğrenen genel amaçlı bir dedektördür.

Deep MultiBox.

R-CNN'in aksine, Szegedy ve ark., Selective Search kullanmak yerine ilgi bölgelerini (regions of interest) tahmin etmek için bir convolutional neural network eğitir [8]. MultiBox, confidence tahminini tek bir sınıf tahminiyle değiştirerek tek nesne tespiti de gerçekleştirebilir. Ancak MultiBox genel nesne tespiti gerçekleştiremez ve hâlâ daha büyük bir tespit pipeline'ının yalnızca bir parçasıdır; ek görüntü yaması (image patch) sınıflandırması gerektirir. Hem YOLO hem de MultiBox, bir görüntüdeki bounding box'ları tahmin etmek için bir convolutional network kullanır, ancak YOLO eksiksiz bir tespit sistemidir.

OverFeat.

Sermanet ve ark., lokalizasyon gerçekleştirmek üzere bir convolutional neural network eğitir ve bu lokalizatörü tespit gerçekleştirecek şekilde uyarlar [32]. OverFeat, sliding window tespitini verimli bir şekilde gerçekleştirir ancak yine de ayrık (disjoint) bir sistemdir. OverFeat, tespit performansı için değil lokalizasyon için optimize edilir. DPM gibi, lokalizatör bir tahmin yaparken yalnızca yerel (local) bilgiyi görür. OverFeat, küresel bağlam (global context) hakkında akıl yürütemez ve bu nedenle tutarlı tespitler üretmek için önemli miktarda post-processing gerektirir.

MultiGrasp.

Çalışmamız, tasarım açısından Redmon ve ark. [27] tarafından yapılan grasp (kavrama) tespiti çalışmasına benzemektedir. Bounding box tahminine yönelik grid yaklaşımımız, grasp'lara regresyon için kullanılan MultiGrasp sistemine dayanmaktadır. Ancak grasp tespiti, nesne tespitinden çok daha basit bir görevdir. MultiGrasp'in yalnızca tek bir nesne içeren bir görüntü için tek bir kavranabilir (graspable) bölge tahmin etmesi gerekir. Nesnenin boyutunu, konumunu veya sınırlarını tahmin etmesi ya da sınıfını öngörmesi gerekmez; yalnızca kavramaya uygun bir bölge bulması yeterlidir. YOLO ise bir görüntüdeki birden çok sınıftan birden çok nesne için hem bounding box'ları hem de sınıf olasılıklarını tahmin eder.

4. Deneyler

İlk olarak, YOLO'yu PASCAL VOC 2007 üzerinde diğer gerçek zamanlı tespit sistemleriyle karşılaştırıyoruz. YOLO ile R-CNN varyantları arasındaki farkları anlamak için, YOLO ve R-CNN'in en yüksek performanslı versiyonlarından biri olan Fast R-CNN'in [14] VOC 2007 üzerinde yaptığı hataları inceliyoruz. Farklı hata profillerine dayanarak, YOLO'nun Fast R-CNN tespitlerini yeniden puanlamak (rescore) ve arka plan yanlış pozitiflerinden kaynaklanan hataları azaltmak için kullanılabileceğini ve bunun önemli bir performans artışı sağladığını gösteriyoruz. Ayrıca

VOC 2012 sonuçlarını sunuyor ve mAP'i mevcut en gelişmiş (state-of-the-art) yöntemlerle karşılaştırıyoruz. Son olarak, YOLO'nun iki sanat eseri veri kümesi üzerinde diğer dedektörlere kıyasla yeni alanlara daha iyi genelleme yaptığını gösteriyoruz.

4.1. Diğer Gerçek Zamanlı Sistemlerle Karşılaştırma

Nesne tespitindeki birçok araştırma çalışması, standart tespit pipeline'larını hızlandırmaya odaklanır [5] [38] [31] [14] [17] [28]. Ancak yalnızca Sadeghi ve ark., gerçek anlamda gerçek zamanlı çalışan (saniyede 30 kare veya daha iyi) bir tespit sistemi ortaya koyar [31]. YOLO'yu, 30Hz veya 100Hz hızında çalışan DPM'nin GPU implementasyonu ile karşılaştırıyoruz. Diğer çalışmalar gerçek zamanlı eşiğe ulaşmasa da, nesne tespit sistemlerinde mevcut olan doğruluk-performans ödünleşimlerini (tradeoffs) incelemek için görelî mAP ve hızlarını da karşılaştırıyoruz.

Fast YOLO, PASCAL üzerindeki en hızlı nesne tespit yöntemidir; bildiğimiz kadarıyla, günümüzde var olan en hızlı nesne dedektörüdür. %52,7 mAP değeriyle, gerçek zamanlı tespit üzerine yapılmış önceki çalışmalardan iki kattan daha doğrudur. YOLO ise gerçek zamanlı performansı korurken mAP'i %63,4'e çıkarır.

Ayrıca YOLO'yu VGG-16 kullanarak da eğitiyoruz. Bu model daha doğrudur ancak YOLO'dan da belirgin biçimde daha yavaştır. VGG-16'ya dayanan diğer tespit sistemleriyle karşılaştırma için faydalıdır, ancak gerçek zamandan daha yavaş olduğu için makalenin geri kalanı daha hızlı modellerimize odaklanmaktadır.

Gerçek Zamanlı Dedektörler	Eğitim	mAP	FPS
100Hz DPM [31]	2007	16.0	100
30Hz DPM [31]	2007	26.1	30
Fast YOLO	2007+2012	52.7	155
YOLO	2007+2012	63.4	45
Gerçek Zamandan Yavaş (Less Than Real-Time)			
Fastest DPM [38]	2007	30.4	15
R-CNN Minus R [20]	2007	53.5	6
Fast R-CNN [14]	2007+2012	70.0	0.5
Faster R-CNN VGG-16 [28]	2007+2012	73.2	7
Faster R-CNN ZF [28]	2007+2012	62.1	18
YOLO VGG-16	2007+2012	66.4	21

Tablo 1: PASCAL VOC 2007 Üzerinde Gerçek Zamanlı Sistemler. Hızlı dedektörlerin performansını ve hızını karşılaştırmaktadır. Fast YOLO, PASCAL VOC tespiti için şimdiye kadar kaydedilen en hızlı dedektördür ve yine de diğer herhangi bir gerçek zamanlı dedektörden iki kat daha doğrudur. YOLO, hızlı versiyondan 10 mAP daha doğrudur ve hız bakımından hâlâ gerçek zamanın oldukça üzerindedir.

Fastest DPM, mAP'ten fazla ödün vermeden DPM'yi etkili bir şekilde hızlandırır, ancak yine de gerçek zamanlı performansı 2 kat farkla kaçırır [38]. Ayrıca DPM'nin, sinir ağı (neural network) yaklaşımlarına kıyasla tespitte nispeten düşük doğruluğuyla sınırlıdır.

R-CNN minus R, Selective Search'ü statik bounding box önerileriyle değiştirir [20]. R-CNN'den çok daha hızlı olsa da, yine de gerçek zamandan geridedir ve iyi önerilere sahip olmamaktan dolayı doğrulukta ciddi bir kayıp yaşar.

Fast R-CNN, R-CNN'in sınıflandırma aşamasını hızlandırır, ancak yine de bounding box önerileri üretmek için görüntü başına yaklaşık 2 saniye sürebilen selective search'e dayanır. Bu nedenle yüksek mAP'e sahiptir, ancak 0,5 fps ile gerçek zamandan hâlâ oldukça uzaktır.

Yakın zamanda geliştirilen Faster R-CNN, Szegedy ve ark. [8] çalışmasına benzer şekilde, bounding box önermek için selective search'ü bir sinir ağıyla değiştirir. Testlerimizde, en doğru modelleri 7 fps'ye ulaşırken, daha küçük ve daha az doğru bir versiyonu 18 fps'de çalışır. Faster R-CNN'in VGG-16 versiyonu 10 mAP daha yüksektir, ancak YOLO'dan 6 kat daha yavaştır. Zeiler-Fergus Faster R-CNN ise YOLO'dan yalnızca 2,5 kat daha yavaştır ancak daha az doğrudur.

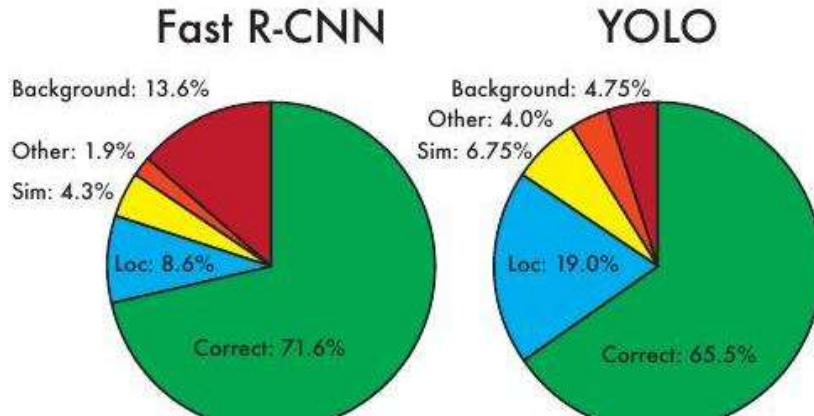
4.2. VOC 2007 Hata Analizi

YOLO ile en gelişmiş dedektörler arasındaki farkları daha ayrıntılı incelemek için, VOC 2007 üzerindeki sonuçların detaylı bir dökümüne bakıyoruz. Fast R-CNN, PASCAL üzerindeki en yüksek performanslı dedektörlerden biri olduğundan ve tespitleri kamuya açık olduğundan, YOLO'yu Fast R-CNN ile karşılaştırıyoruz.

Hoiem ve ark.'nın [19] metodolojisini ve araçlarını kullanıyoruz. Her sınıf için, test zamanında o sınıfa ait ilk N tahmine bakıyoruz. Her tahmin ya doğrudur ya da hata tipine göre şu şekilde sınıflandırılır:

- Correct (Doğru): doğru sınıf ve IOU > 0,5
- Localization (Lokalizasyon): doğru sınıf, $0,1 < \text{IOU} < 0,5$
- Similar (Benzer): sınıf benzer, IOU > 0,1
- Other (Diğer): sınıf yanlış, IOU > 0,1
- Background (Arka plan): herhangi bir nesne için IOU < 0,1

Şekil 4, tüm 20 sınıf üzerinden ortalanan her hata tipinin dökümünü göstermektedir.



Şekil 4: Hata Analizi: Fast R-CNN vs. YOLO. Bu grafikler, çeşitli kategoriler için ilk N tespit içindeki lokalizasyon ve arka plan hatalarının yüzdesini göstermektedir (N = o kategorideki nesne sayısı).

YOLO, nesneleri doğru şekilde lokalize etmekte zorlanır. Lokalizasyon hataları, YOLO'nun hatalarının diğer tüm kaynakların toplamından daha büyük bir kısmını oluşturur. Fast R-CNN çok daha az lokalizasyon hatası yapar ancak çok daha fazla arka plan hatası yapar. En üst tespitlerinin %13,6'sı, herhangi bir nesne içermeyen yanlış pozitiflerdir. Fast R-CNN, YOLO'ya kıyasla arka plan tespiti yapma olasılığı neredeyse 3 kat daha yüksektir.

4.3. Fast R-CNN ve YOLO'yu Birleştirmek

YOLO, Fast R-CNN'e kıyasla çok daha az arka plan hatası yapar. Fast R-CNN'den gelen arka plan tespitlerini elemek için YOLO'yu kullanarak performansta önemli bir artış elde ediyoruz. R-CNN'in tahmin ettiği her bounding box için, YOLO'nun benzer bir box tahmin edip etmediğini kontrol ediyoruz. Eğer ediyorsa, YOLO tarafından tahmin edilen olasılığa ve iki box arasındaki örtüşmeye (overlap) dayanarak o tahmine bir artış (boost) veriyoruz.

En iyi Fast R-CNN modeli, VOC 2007 test kümesinde %71,8'lik bir mAP elde eder. YOLO ile birleştirildiğinde, mAP'i %3,2 artarak %75,0'a çıkar. Ayrıca en iyi Fast R-CNN modelini, Fast R-CNN'in çeşitli diğer versiyonlarıyla birleştirmeyi de denedik. Bu ensemble'lar, mAP'te %0,3 ile %0,6 arasında küçük artışlar sağladı; ayrıntılar için bkz. Tablo 2.

	mAP	Combined	Gain
Fast R-CNN	71.8	-	-
Fast R-CNN (2007 verisi)	66.9	72.4	.6
Fast R-CNN (VGG-M)	59.2	72.4	.6
Fast R-CNN (CaffeNet)	57.1	72.1	.3
YOLO	63.4	75.0	3.2

Tablo 2: VOC 2007 Üzerinde Model Birleştirme Deneyleri. Çeşitli modelleri Fast R-CNN'in en iyi versiyonuyla birleştirmenin etkisini inceliyoruz. Fast R-CNN'in diğer versiyonları yalnızca küçük bir fayda sağlarken, YOLO önemli bir performans artışı sağlamaktadır.

YOLO'dan gelen artış, yalnızca model ensembling'in bir yan ürünü değildir; çünkü Fast R-CNN'in farklı versiyonlarını birleştirmenin sağladığı fayda oldukça azdır. Aksine, YOLO'nun test zamanında farklı türde hatalar yapması, tam olarak onu Fast R-CNN'in performansını artırmada bu kadar etkili kılan şeydir.

Ne yazık ki, her modeli ayrı ayrı çalıştırıp sonuçları birleştirdiğimiz için bu kombinasyon, YOLO'nun hızından faydalanamaz. Ancak YOLO o kadar hızlıdır ki, Fast R-CNN ile karşılaştırıldığında önemli bir ek hesaplama süresi getirmez.

4.4. VOC 2012 Sonuçları

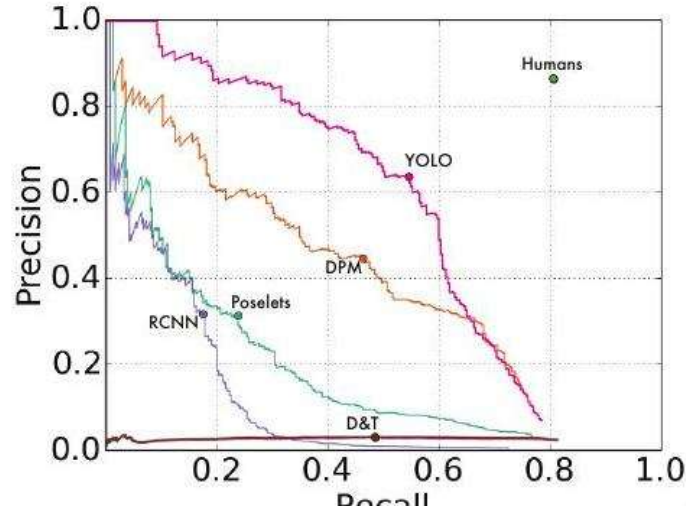
VOC 2012 test kümesinde YOLO, %57,9 mAP puanı alır. Bu, mevcut en gelişmiş yöntemlerden düşüktür ve VGG-16 kullanan orijinal R-CNN'e daha yakındır, bkz. Tablo 3. Sistemimiz, en yakın rakiplerine kıyasla küçük nesnelerle zorlanmaktadır. bottle, sheep ve tv/monitor gibi kategorilerde YOLO, R-CNN veya Feature Edit'ten %8-10 daha düşük puan alır. Ancak cat ve train gibi diğer kategorilerde YOLO daha yüksek performans elde eder.

Birleştirilmiş Fast R-CNN + YOLO modelimiz, en yüksek performanslı tespit yöntemlerinden biridir. Fast R-CNN, YOLO ile birleşiminden %2,3'lük bir iyileşme elde eder ve bu da onu kamuya açık liderlik tablosunda (leaderboard) 5 sıra yukarı taşır.

4.5. Genellenebilirlik: Sanat Eserlerinde İnsan Tespiti

Nesne tespiti için akademik veri kümeleri, eğitim ve test verisini aynı dağılımdan alır. Gerçek dünya uygulamalarında olası tüm kullanım senaryolarını öngörmek zordur ve test verisi, sistemin daha önce gördüğü verilerden sapabilir [3]. YOLO'yu, sanat eserlerinde insan tespitini test etmek için kullanılan iki veri kümesi olan Picasso Dataset [12] ve People-Art Dataset [3] üzerinde diğer tespit sistemleriyle karşılaştırıyoruz.

Şekil 5, YOLO ile diğer tespit yöntemleri arasındaki karşılaştırmalı performansı göstermektedir. Referans olması için, tüm modellerin yalnızca VOC 2007 verisi üzerinde eğitildiği person sınıfı için VOC 2007 tespit AP değerini veriyoruz. Picasso üzerinde modeller VOC 2012 üzerinde, People-Art üzerinde ise VOC 2010 üzerinde eğitilmiştir.



(a) Picasso Dataset precision-recall eğrileri.

	VOC 2007	Picasso		People-Art
	AP	AP	Best F1	AP
YOLO	59.2	53.3	0.590	45
R-CNN	54.2	10.4	0.226	26
DPM	43.2	37.8	0.458	32
Poselets [2]	36.5	17.8	0.271	-
D&T [4]	-	1.9	0.051	-

(b) VOC 2007, Picasso ve People-Art veri kümeleri üzerindeki nicel (quantitative) sonuçlar. Picasso Dataset hem AP hem de en iyi F1 score üzerinden değerlendirme yapar.

Şekil 5: Picasso ve People-Art veri kümeleri üzerindeki genelleme sonuçları.

R-CNN, VOC 2007 üzerinde yüksek AP değerine sahiptir. Ancak sanat eserlerine uygulandığında R-CNN'in performansı belirgin şekilde düşer. R-CNN, bounding box önerileri için doğal görüntülere göre ayarlanmış (tuned) Selective Search kullanır. R-CNN'deki sınıflandırıcı adımı yalnızca küçük bölgeleri görür ve iyi önerilere ihtiyaç duyar.

DPM, sanat eserlerine uygulandığında AP değerini iyi korur. Önceki çalışmalar, DPM'nin nesnelerin şekli ve düzenine (layout) ilişkin güçlü uzamsal modellere sahip olması nedeniyle iyi performans gösterdiğini öne sürmektedir. DPM, R-CNN kadar bozulmasa da, daha düşük bir AP'den başlar.

YOLO, VOC 2007 üzerinde iyi performans gösterir ve sanat eserlerine uygulandığında AP değeri diğer yöntemlere kıyasla daha az düşer. YOLO, DPM gibi, nesnelerin boyutunu ve şeklini, ayrıca nesneler arasındaki ilişkileri ve nesnelerin genellikle nerede görüldüğünü modeller. Sanat eserleri ve doğal görüntüler piksel düzeyinde çok farklıdır, ancak nesnelerin boyutu ve şekli açısından benzerdir; bu nedenle YOLO yine de iyi bounding box'lar ve tespitler tahmin edebilir.

- [3] H. Cai, Q. Wu, T. Corradi, and P. Hall. The cross-depiction problem: Computer vision algorithms for recognising objects in artwork and in photographs. arXiv preprint arXiv:1505.00110, 2015.
- [4] N. Dalal and B. Triggs. Histograms of oriented gradients for human detection. In Computer Vision and Pattern Recognition, 2005. CVPR 2005. IEEE Computer Society Conference on, volume 1, pages 886–893. IEEE, 2005.
- [5] T. Dean, M. Ruzon, M. Segal, J. Shlens, S. Vijayanarasimhan, J. Yagnik, et al. Fast, accurate detection of 100,000 object classes on a single machine. In Computer Vision and Pattern Recognition (CVPR), 2013 IEEE Conference on, pages 1814–1821. IEEE, 2013.
- [6] J. Donahue, Y. Jia, O. Vinyals, J. Hoffman, N. Zhang, E. Tzeng, and T. Darrell. Decaf: A deep convolutional activation feature for generic visual recognition. arXiv preprint arXiv:1310.1531, 2013.
- [7] J. Dong, Q. Chen, S. Yan, and A. Yuille. Towards unified object detection and semantic segmentation. In Computer Vision–ECCV 2014, pages 299–314. Springer, 2014.
- [8] D. Erhan, C. Szegedy, A. Toshev, and D. Anguelov. Scalable object detection using deep neural networks. In Computer Vision and Pattern Recognition (CVPR), 2014 IEEE Conference on, pages 2155–2162. IEEE, 2014.
- [9] M. Everingham, S. M. A. Eslami, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman. The pascal visual object classes challenge: A retrospective. International Journal of Computer Vision, 111(1):98–136, Jan. 2015.
- [10] P. F. Felzenszwalb, R. B. Girshick, D. McAllester, and D. Ramanan. Object detection with discriminatively trained part based models. IEEE Transactions on Pattern Analysis and Machine Intelligence, 32(9):1627–1645, 2010.
- [11] S. Gidaris and N. Komodakis. Object detection via a multi-region & semantic segmentation-aware CNN model. CoRR, abs/1505.01749, 2015.
- [12] S. Ginosar, D. Haas, T. Brown, and J. Malik. Detecting people in cubist art. In Computer Vision-ECCV 2014 Workshops, pages 101–116. Springer, 2014.
- [13] R. Girshick, J. Donahue, T. Darrell, and J. Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In Computer Vision and Pattern Recognition (CVPR), 2014 IEEE Conference on, pages 580–587. IEEE, 2014.
- [14] R. B. Girshick. Fast R-CNN. CoRR, abs/1504.08083, 2015.
- [15] S. Gould, T. Gao, and D. Koller. Region-based segmentation and object detection. In Advances in neural information processing systems, pages 655–663, 2009.
- [16] B. Hariharan, P. Arbelaez, R. Girshick, and J. Malik. Simultaneous detection and segmentation. In Computer Vision–ECCV 2014, pages 297–312. Springer, 2014.
- [17] K. He, X. Zhang, S. Ren, and J. Sun. Spatial pyramid pooling in deep convolutional networks for visual recognition. arXiv preprint arXiv:1406.4729, 2014.
- [18] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. arXiv preprint arXiv:1207.0580, 2012.
- [19] D. Hoiem, Y. Chodpathumwan, and Q. Dai. Diagnosing error in object detectors. In Computer Vision–ECCV 2012, pages 340–353. Springer, 2012.
- [20] K. Lenc and A. Vedaldi. R-cnn minus r. arXiv preprint arXiv:1506.06981, 2015.
- [21] R. Lienhart and J. Maydt. An extended set of haar-like features for rapid object detection. In Image Processing. 2002. Proceedings. 2002 International Conference on, volume 1, pages I–900. IEEE, 2002.
- [22] M. Lin, Q. Chen, and S. Yan. Network in network. CoRR, abs/1312.4400, 2013.
- [23] D. G. Lowe. Object recognition from local scale-invariant features. In Computer vision, 1999. The proceedings of the seventh IEEE international conference on, volume 2, pages 1150–1157. IEEE, 1999.
- [24] D. Mishkin. Models accuracy on imagenet 2012 val. <https://github.com/BVLC/caffe/wiki/Models-accuracy-on-ImageNet-2012-val>. Accessed: 2015-10-2.
- [25] C. P. Papageorgiou, M. Oren, and T. Poggio. A general framework for object detection. In Computer vision, 1998. sixth international conference on, pages 555–562. IEEE, 1998.
- [26] J. Redmon. Darknet: Open source neural networks in c. <http://pjreddie.com/darknet/>, 2013–2016.

- [27] J. Redmon and A. Angelova. Real-time grasp detection using convolutional neural networks. CoRR, abs/1412.3128, 2014.
- [28] S. Ren, K. He, R. Girshick, and J. Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. arXiv preprint arXiv:1506.01497, 2015.
- [29] S. Ren, K. He, R. B. Girshick, X. Zhang, and J. Sun. Object detection networks on convolutional feature maps. CoRR, abs/1504.06066, 2015.
- [30] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei. ImageNet Large Scale Visual Recognition Challenge. International Journal of Computer Vision (IJCV), 2015.
- [31] M. A. Sadeghi and D. Forsyth. 30hz object detection with dpm v5. In Computer Vision–ECCV 2014, pages 65–79. Springer, 2014.
- [32] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. LeCun. Overfeat: Integrated recognition, localization and detection using convolutional networks. CoRR, abs/1312.6229, 2013.
- [33] Z. Shen and X. Xue. Do more dropouts in pool5 feature maps for better object detection. arXiv preprint arXiv:1409.6911, 2014.
- [34] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. CoRR, abs/1409.4842, 2014.
- [35] J. R. Uijlings, K. E. van de Sande, T. Gevers, and A. W. Smeulders. Selective search for object recognition. International journal of computer vision, 104(2):154–171, 2013.
- [36] P. Viola and M. Jones. Robust real-time object detection. International Journal of Computer Vision, 4:34–47, 2001.
- [37] P. Viola and M. J. Jones. Robust real-time face detection. International journal of computer vision, 57(2):137–154, 2004.
- [38] J. Yan, Z. Lei, L. Wen, and S. Z. Li. The fastest deformable part model for object detection. In Computer Vision and Pattern Recognition (CVPR), 2014 IEEE Conference on, pages 2497–2504. IEEE, 2014.
- [39] C. L. Zitnick and P. Dollar. Edge boxes: Locating object proposals from edges. In Computer Vision–ECCV 2014, pages 391–405. Springer, 2014.

Tablo 3: PASCAL VOC 2012 Liderlik Tablosu.

YOLO, 6 Kasım 2015 itibarıyla tam comp4 (dışarıdan veriye izin verilen) kamuya açık liderlik tablosuyla karşılaştırılmıştır. Çeşitli tespit yöntemleri için ortalama hassasiyet (mean average precision) ve sınıf başına ortalama hassasiyet gösterilmektedir. YOLO, tek gerçek zamanlı dedektördür. Fast R-CNN + YOLO, Fast R-CNN'e göre %2,3'lük bir artışla dördüncü en yüksek puanlı yöntemdir. Sınıf adları (aero, bike, bird vb.) PASCAL VOC veri kümesindeki orijinal İngilizce hâliyle bırakılmıştır.

Method	mAP	aero	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
MR CNN MORE DATA [11]	73.9	85.5	82.9	76.6	57.8	62.7	79.4	77.2	86.6	55.0	79.1	62.2	87.0	83.4	84.7	78.9	45.3	73.4	65.8	80.3	74.0
HyperNet VGG	71.4	84.2	78.5	73.6	55.6	53.7	78.7	79.8	87.7	49.6	74.9	52.1	86.0	81.7	83.3	81.8	48.6	73.5	59.4	79.9	65.7
HyperNet SP	71.3	84.1	78.3	73.3	55.5	53.6	78.6	79.6	87.5	49.5	74.9	52.1	85.6	81.6	83.2	81.6	48.4	73.2	59.3	79.7	65.6
Fast R-CNN + YOLO	70.7	83.4	78.5	73.5	55.8	43.4	79.1	73.1	89.4	49.4	75.5	57.0	87.5	80.9	81.0	74.7	41.8	71.5	68.5	82.1	67.2
MR CNN S CNN [11]	70.7	85.0	79.6	71.5	55.3	57.7	76.0	73.9	84.6	50.5	74.3	61.7	85.5	79.9	81.7	76.4	41.0	69.0	61.2	77.7	72.1
Faster R-CNN [28]	70.4	84.9	79.8	74.3	53.9	49.8	77.5	75.9	88.5	45.6	77.1	55.3	86.9	81.7	80.9	79.6	40.1	72.6	60.9	81.2	61.5
DEEP ENS COCO	70.1	84.0	79.4	71.6	51.9	51.1	74.1	72.1	88.6	48.3	73.4	57.8	86.1	80.0	80.7	70.4	46.6	69.6	68.8	75.9	71.4
NoC [29]	68.8	82.8	79.0	71.6	52.3	53.7	74.1	69.0	84.9	46.9	74.3	53.1	85.0	81.3	79.5	72.2	38.9	72.4	59.5	76.7	68.1
Fast R-CNN [14]	68.4	82.3	78.4	70.8	52.3	38.7	77.8	71.6	89.3	44.2	73.0	55.0	87.5	80.5	80.8	72.0	35.1	68.3	65.7	80.4	64.2
UMICH FGS STRUCT	66.4	82.9	76.1	64.1	44.6	49.4	70.3	71.2	84.6	42.7	68.6	55.8	82.7	77.1	79.9	68.7	41.4	69.0	60.0	72.0	66.2
NUS NIN C2000 [7]	63.8	80.2	73.8	61.9	43.7	43.0	70.3	67.6	80.7	41.9	69.7	51.7	78.2	75.2	76.9	65.1	38.6	68.3	58.0	68.7	63.3
BabyLearning [7]	63.2	78.0	74.2	61.3	45.7	42.7	68.2	66.8	80.2	40.6	70.0	49.8	79.0	74.5	77.9	64.0	35.3	67.9	55.7	68.7	62.6
NUS NIN	62.4	77.9	73.1	62.6	39.5	43.3	69.1	66.4	78.9	39.1	68.1	50.0	77.2	71.3	76.1	64.7	38.4	66.9	56.2	66.9	62.7
R-CNN VGG BB [13]	62.4	79.6	72.7	61.9	41.2	41.9	65.9	66.4	84.6	38.5	67.2	46.7	82.0	74.8	76.0	65.2	35.6	65.4	54.2	67.4	60.3
R-CNN VGG [13]	59.2	76.8	70.9	56.6	37.5	36.9	62.9	63.6	81.1	35.7	64.3	43.9	80.4	71.6	74.0	60.0	30.8	63.4	52.0	63.5	58.7
YOLO	57.9	77.0	67.2	57.7	38.3	22.7	68.3	55.9	81.4	36.2	60.8	48.5	77.2	72.3	71.3	63.5	28.9	52.2	54.8	73.9	50.8
Feature Edit [33]	56.3	74.6	69.1	54.4	39.1	33.1	65.2	62.7	69.7	30.8	56.0	44.6	70.0	64.4	71.1	60.2	33.3	61.3	46.4	61.7	57.8
R-CNN BB [13]	53.3	71.8	65.8	52.0	34.1	32.6	59.6	60.0	69.8	27.6	52.0	41.7	69.6	61.3	68.3	57.8	29.6	57.8	40.9	59.3	54.1
SDS [16]	50.7	69.7	58.4	48.5	28.3	28.8	61.3	57.5	70.8	24.1	50.7	35.9	64.9	59.1	65.8	57.1	26.0	58.8	38.6	58.9	50.7
R-CNN [13]	49.6	68.1	63.8	46.1	29.4	27.9	56.6	57.0	65.9	26.5	48.7	39.5	66.2	57.3	65.4	53.2	26.2	54.5	38.1	50.6	51.6